

DBMS PROGRAMS

Q.No: 1

Scenario: Student Academic Management System

A college is developing a Student Academic Management System to efficiently store and

manage student personal and academic records.

The system initially requires two core tables:

- **One table to store student personal details such as registration number, name, date of**

birth, address, and city.

- **Another table to maintain student marks in three subjects along with their total score.**

As the system evolves, new requirements arise:

- **The academic team decides that, in addition to total marks, the system should also**

maintain the average marks for each student. The database structure must be modified accordingly.

- **The administration department wants to analyze student age data by calculating the**

difference in months between each student's date of birth and the current date.

- **Due to privacy concerns, the institution decides to remove the address field from the**

student records.

- **For reporting purposes, the system should include a stored procedure that calculates**

the sum and average of marks for all students.

- **Finally, the application team needs to integrate the database with a Java-based application, requiring a program that establishes a connection to the database using**

JDBC.

Ans).

1. Table Creation and Data Insertion (5 Marks)

Student Table

SQL

```
CREATE TABLE Student (  
    Reg_no NUMBER PRIMARY KEY,  
    Name VARCHAR2(50) NOT NULL,  
    Dob DATE,  
    Address VARCHAR2(100),  
    City VARCHAR2(50)  
);
```

Marks Table

SQL

```
CREATE TABLE Marks (  
    Reg_no NUMBER,  
    Sub1 NUMBER,  
    Sub2 NUMBER,  
    Sub3 NUMBER,  
    Total NUMBER,  
    FOREIGN KEY (reg_no) REFERENCES Student(reg_no)  
);
```

Insert Data

SQL

```
INSERT INTO Student VALUES (101, 'Ravi', TO_DATE('2003-05-12','YYYY-MM-DD'), 'MG  
Road', 'Chennai');
```

```
INSERT INTO Student VALUES (102, 'Priya', TO_DATE('2002-11-20','YYYY-MM-DD'), 'Anna  
Nagar', 'Chennai');
```

```
INSERT INTO Marks VALUES (101, 85, 90, 88, 263);
```

INSERT INTO Marks VALUES (102, 75, 80, 70, 225);

2. Constraints and Data Types (5 Marks)

PRIMARY KEY → reg_no

FOREIGN KEY → Links Student & Marks table

NOT NULL → Name cannot be empty

NUMBER → For marks and reg_no

VARCHAR2 → For text fields

DATE → For DOB

3. Required Queries (6 Marks)

(a) Add Average Column

SQL

ALTER TABLE Marks ADD avg_marks NUMBER;

(b) Update Average Marks

SQL

UPDATE Marks

SET avg_marks = total / 3;

(c) Calculate Age in Months

SQL

SELECT name, MONTHS_BETWEEN(SYSDATE, dob) AS age_in_months

FROM Student;

(d) Remove Address Column

SQL

ALTER TABLE Student DROP COLUMN address;

4. Stored Procedure (7 Marks)

SQL

CREATE OR REPLACE PROCEDURE calc_total_avg IS

 Total_sum NUMBER := 0;

 Avg_marks NUMBER := 0;

```

BEGIN

    SELECT SUM(total), AVG(total)

    INTO total_sum, avg_marks

    FROM Marks;


    DBMS_OUTPUT.PUT_LINE('Total Marks of all students: ' || total_sum);

    DBMS_OUTPUT.PUT_LINE('Average Marks of all students: ' || avg_marks);

END;

/

```

Execute Procedure

SQL

```

BEGIN

    Calc_total_avg;

END;

/

```

5. JDBC Program (7 Marks)

Java

```

Import java.sql.*;

Public class StudentDB {

    Public static void main(String[] args) {

        Try {

            Class.forName("oracle.jdbc.driver.OracleDriver");

            Connection con = DriverManager.getConnection(

                "jdbc:oracle:thin:@localhost:1521:xe",

                "system",

                "password"

            );

            System.out.println("Connected Successfully!");

```

```

Statement stmt = con.createStatement();

ResultSet rs = stmt.executeQuery("SELECT * FROM Student");

While(rs.next()) {

    System.out.println(

        Rs.getInt(1) + " " +

        Rs.getString(2) + " " +

        Rs.getDate(3)

    );

}

Con.close();

} catch(Exception e) {

    System.out.println(e);

}

}

}

```

Q.No: 2

Scenario: Student Examination Record System

A college is developing a Student Examination Record System to maintain student personal details and their academic performance.

Initially, the system includes:

- **A table to store student information such as registration number, name, date of birth, address, and city.**
- **Another table to store marks obtained by students in three subjects along with their total score.**

As the system is actively used by the examination department, several operational needs

arise:

- The faculty wants to identify students who have performed well by retrieving records

of those whose total marks fall within a specific high-performance range (between 250 and 300).

- At a later stage, the marks table is found to be redundant due to system restructuring,

and it needs to be removed completely from the database.

- During data verification, it is discovered that a particular student record (with a specific registration number) was entered incorrectly, and the system must support

removal of that specific record.

- For administrative purposes, the staff also requires a way to filter and view student

records where the student names start with a particular letter (for example, 'A'), to simplify searching and reporting.

Ans).

1. Table Creation and Data Insertion (5 Marks)

Student Table

SQL

```
CREATE TABLE Student (
```

```
    Reg_no NUMBER PRIMARY KEY,
```

```
    Name VARCHAR2(50) NOT NULL,
```

```
    Dob DATE,
```

```
    Address VARCHAR2(100),
```

```
    City VARCHAR2(50)
```

```
);
```

Marks Table

SQL

```
CREATE TABLE Marks (  
    Reg_no NUMBER,  
    Sub1 NUMBER,  
    Sub2 NUMBER,  
    Sub3 NUMBER,  
    Total NUMBER,  
    FOREIGN KEY (reg_no) REFERENCES Student(reg_no)  
);
```

Insert Sample Data

SQL

```
INSERT INTO Student VALUES (101, 'Arun', TO_DATE('2003-04-10','YYYY-MM-DD'), 'MG  
Road', 'Chennai');
```

```
INSERT INTO Student VALUES (102, 'Anitha', TO_DATE('2002-09-15','YYYY-MM-DD'),  
'Anna Nagar', 'Chennai');
```

```
INSERT INTO Student VALUES (103, 'Bala', TO_DATE('2003-01-20','YYYY-MM-DD'), 'T  
Nagar', 'Chennai');
```

```
INSERT INTO Marks VALUES (101, 85, 90, 88, 263);
```

```
INSERT INTO Marks VALUES (102, 80, 85, 82, 247);
```

```
INSERT INTO Marks VALUES (103, 70, 75, 80, 225);
```

2. Constraints and Data Types (5 Marks)

PRIMARY KEY → reg_no uniquely identifies each student

FOREIGN KEY → Maintains relationship between Student and Marks

NOT NULL → Ensures name is mandatory

NUMBER → Used for numeric values (marks, reg_no)

VARCHAR2 → Used for text fields

DATE → Used for storing date of birth

3. Required Queries (6 Marks)

(a) Retrieve High-Performance Students (250–300)

SQL

```
SELECT s.reg_no, s.name, m.total  
FROM Student s  
JOIN Marks m ON s.reg_no = m.reg_no  
WHERE m.total BETWEEN 250 AND 300;
```

(b) Remove Marks Table Completely

SQL

```
DROP TABLE Marks;
```

(c) Delete Specific Student Record

SQL

```
DELETE FROM Student  
WHERE reg_no = 101;
```

(d) Retrieve Students Whose Names Start with 'A'

SQL

```
SELECT * FROM Student  
WHERE name LIKE 'A%';
```

4 .Explanation (Extra Scoring Point)

BETWEEN 250 AND 300 → Filters top-performing students

DROP TABLE → Deletes entire table structure and data

DELETE → Removes specific incorrect record

LIKE 'A%' → Pattern matching for names starting with 'A'

Q.No: 3

Scenario: Bookstore and Customer Management System

A bookstore is developing a Bookstore and Customer Management System to handle book

inventory and customer information efficiently.

Initially, the system is designed with:

- A table to store book details such as book name, author, price, and quantity available.
- Another table to maintain customer information including customer ID, name, address, phone number, and PAN number.

As the system is implemented and used in daily operations, several requirements arise:

Operational Changes and Data Management

- Due to periodic maintenance and data refresh requirements, the customer records need

to be cleared completely while preserving the table structure for future use.

Book Inventory Queries

- The store manager wants to identify books that are priced at a specific value (₹200)

and retrieve the corresponding author details.

- For pricing analysis, the system should allow retrieval of books whose price falls within a certain range (between ₹175 and ₹250).

- For promotional purposes, the management also needs to filter and display books

where the author's name starts with a specific letter (such as 'K').

Additional Logic Requirement

- As part of backend processing, the system should include a PL/SQL program that determines the largest of two given numbers, which can be reused in various business logic scenarios.

Ans).

1. Table Creation and Data Insertion (5 Marks)

Book Table

SQL

CREATE TABLE Book (

Book_id NUMBER PRIMARY KEY,

```
Book_name VARCHAR2(100),  
Author VARCHAR2(50),  
Price NUMBER,  
Quantity NUMBER  
);
```

Customer Table

SQL

```
CREATE TABLE Customer (  
    Cust_id NUMBER PRIMARY KEY,  
    Name VARCHAR2(50),  
    Address VARCHAR2(100),  
    Phone VARCHAR2(15),  
    Pan_no VARCHAR2(20)  
);
```

Insert Sample Data

SQL

```
INSERT INTO Book VALUES (1, 'DBMS Concepts', 'Kumar', 200, 10);
```

```
INSERT INTO Book VALUES (2, 'Java Basics', 'Kiran', 180, 15);
```

```
INSERT INTO Book VALUES (3, 'Python Guide', 'Arun', 250, 8);
```

```
INSERT INTO Customer VALUES (101, 'Ravi', 'Chennai', '9876543210', 'ABCDE1234F');
```

```
INSERT INTO Customer VALUES (102, 'Priya', 'Chennai', '9123456780', 'PQRSX5678L');
```

2. Constraints and Data Types (5 Marks)

PRIMARY KEY → book_id, cust_id

VARCHAR2 → For names, address, PAN

NUMBER → For price, quantity

Phone as VARCHAR2 → To preserve formatting

Ensures data integrity and uniqueness

3. Required Queries (6 Marks)

(a) Delete All Customer Records (Keep Table)

SQL

```
TRUNCATE TABLE Customer;
```

(b) Retrieve Books with Price = ₹200

SQL

```
SELECT book_name, author
```

```
FROM Book
```

```
WHERE price = 200;
```

(c) Retrieve Books in Price Range (₹175–₹250)

SQL

```
SELECT *
```

```
FROM Book
```

```
WHERE price BETWEEN 175 AND 250;
```

(d) Retrieve Books Where Author Starts with 'K'

SQL

```
SELECT *
```

```
FROM Book
```

```
WHERE author LIKE 'K%';
```

3. PL/SQL Program (7 Marks)

Program to Find Largest of Two Numbers

SQL

```
DECLARE
```

```
    A NUMBER := 10;
```

```
    B NUMBER := 20;
```

```
BEGIN
```

```
    IF a > b THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Largest number is: ' || a);
```

```
ELSE  
    DBMS_OUTPUT.PUT_LINE('Largest number is: ' || b);  
END IF;  
END;  
/
```

4. Explanation (Extra Marks)

TRUNCATE → Removes all records quickly, keeps structure

= → Used for exact price matching

BETWEEN → Filters range values

LIKE 'K%' → Finds authors starting with K

PL/SQL block → Used for backend logic implementation

Q.No 4

Scenario: Student Information and Records Management System

An educational institution is developing a Student Information and Records Management

System to maintain student personal details and academic records.

Initially, the system includes:

- **A table to store student details such as registration number, name, date of birth, address, and city.**
- **Another table to maintain marks obtained by students in different subjects along with their total score.**

Operational Requirements

As the system is used by administrative staff and faculty, several requirements arise:

- **The administration department needs a simple report that provides a list of all student names for general reference and communication purposes.**

- The institution also wants to identify and retrieve complete details of students who are located in a specific city (for example, Chennai) for regional analysis and outreach programs.

- During system restructuring, the marks table becomes unnecessary, and the system must support removal of the entire marks table.

Advanced Processing Requirements

- For efficient data retrieval, the system should include a PL/SQL function that returns the address of a specific student when their registration number is provided.

Application Integration

- A Java-based application is being developed to interact with the database. This application must be capable of:
 - o Establishing a connection to the database using JDBC.
 - o Performing operations such as deleting specific records from the database when required.

Ans).

1. Table Creation and Data Insertion (5 Marks)

Student Table

SQL

```
CREATE TABLE Student (  
    Reg_no NUMBER PRIMARY KEY,  
    Name VARCHAR2(50) NOT NULL,  
    Dob DATE,  
    Address VARCHAR2(100),  
    City VARCHAR2(50)  
);
```

Marks Table

SQL

CREATE TABLE Marks (

Reg_no NUMBER,

Sub1 NUMBER,

Sub2 NUMBER,

Sub3 NUMBER,

Total NUMBER,

FOREIGN KEY (reg_no) REFERENCES Student(reg_no)

);

Insert Sample Data

SQL

INSERT INTO Student VALUES (101, 'Arun', TO_DATE('2003-04-10','YYYY-MM-DD'), 'MG Road', 'Chennai');

INSERT INTO Student VALUES (102, 'Priya', TO_DATE('2002-09-15','YYYY-MM-DD'), 'Anna Nagar', 'Chennai');

INSERT INTO Student VALUES (103, 'Rahul', TO_DATE('2003-01-20','YYYY-MM-DD'), 'KPHB', 'Hyderabad');

INSERT INTO Marks VALUES (101, 85, 90, 88, 263);

INSERT INTO Marks VALUES (102, 80, 85, 82, 247);

INSERT INTO Marks VALUES (103, 70, 75, 80, 225);

2. Constraints and Data Types (5 Marks)

PRIMARY KEY → reg_no uniquely identifies each student

FOREIGN KEY → Links Marks with Student table

NOT NULL → Ensures name must be entered

NUMBER → Used for Ids and marks

VARCHAR2 → Used for text fields

DATE → Used for storing date of birth

3. Queries (6 Marks)

(a) Retrieve All Student Names

SQL

```
SELECT name FROM Student;
```

(b) Retrieve Students from Chennai

SQL

```
SELECT *
```

```
FROM Student
```

```
WHERE city = 'Chennai';
```

(c) Drop Marks Table

SQL

```
DROP TABLE Marks;
```

3. PL/SQL Function (7 Marks)

Function to Get Address Using Registration Number

SQL

```
CREATE OR REPLACE FUNCTION get_address(p_reg_no NUMBER)
```

```
RETURN VARCHAR2
```

```
IS
```

```
    V_address VARCHAR2(100);
```

```
BEGIN
```

```
    SELECT address INTO v_address
```

```
    FROM Student
```

```
    WHERE reg_no = p_reg_no;
```

```
    RETURN v_address;
```

```
END;
```

```
/
```

Call Function

SQL

```
SELECT get_address(101) FROM dual;
```

4. JDBC Program (7 Marks)

Java

```
Import java.sql.*;
```

```
Public class StudentDB {
```

```
    Public static void main(String[] args) {
```

```
        Try {
```

```
            // Load Driver
```

```
            Class.forName("oracle.jdbc.driver.OracleDriver");
```

```
            // Establish Connection
```

```
            Connection con = DriverManager.getConnection(
```

```
                "jdbc:oracle:thin:@localhost:1521:xe",
```

```
                "system",
```

```
                "password"
```

```
            );
```

```
            System.out.println("Connected Successfully!");
```

```
            // Delete a specific record
```

```
            PreparedStatement ps = con.prepareStatement(
```

```
                "DELETE FROM Student WHERE reg_no = ?"
```

```
            );
```

```
            Ps.setInt(1, 101);
```

```
            Ps.executeUpdate();
```

```

        System.out.println("Record Deleted Successfully!");

        Con.close();

    } catch(Exception e) {
        System.out.println(e);
    }
}

```

5. Explanation (Extra Marks)

SELECT name → Retrieves only student names

WHERE city='Chennai' → Filters based on location

DROP TABLE → Removes entire table

Function → Returns specific value (address)

JDBC → Connects Java application with database

PreparedStatement → Used for dynamic and safe queries

Q.No: 5

Scenario: Banking Customer and Loan Management System

A bank is developing a Customer and Loan Management System to efficiently manage

customer records and loan transactions.

Initially, the system is designed with:

- **A table to store customer details such as customer ID, name, address, phone number,**
- and PAN number.**
- **Another table to maintain loan details including loan ID, loan amount, interest rate,**
- and the associated customer ID.**

Business Requirements and Data Analysis

As the banking system becomes operational, several analytical and structural requirements

emerge:

- **The bank management wants to identify customers who are actively engaged with the**

bank by having both loan accounts and savings/current accounts, so that targeted services can be provided.

- **For financial analysis, the system should generate a report that displays the total number of loans and the sum of loan amounts for a specific customer (for example, “LEENA”).**

- **The bank also needs to identify inactive or less-engaged customers who neither hold**

any account nor have taken any loan, in order to design customer engagement strategies.

- **To enhance data tracking, the database structure should be modified to include an**

additional attribute that stores the number of loans associated with each customer.

Application Integration

To automate operations, a Java-based banking application is being developed. This application must:

- **Establish a connection to the database using JDBC.**
- **Enable interaction with the database for performing operations related to customers and loans.**

Ans).

1. Table Creation and Data Insertion (6 Marks)

Customer Table

SQL

```
CREATE TABLE Customer (  
    Cust_id NUMBER PRIMARY KEY,  
    Name VARCHAR2(50) NOT NULL,  
    Address VARCHAR2(100),  
    Phone VARCHAR2(15),  
    Pan_no VARCHAR2(20)  
);
```

Loan Table

SQL

```
CREATE TABLE Loan (  
    Loan_id NUMBER PRIMARY KEY,  
    Loan_amount NUMBER,  
    Interest_rate NUMBER,  
    Cust_id NUMBER,  
    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id)  
);
```

Account Table (for requirement)

SQL

```
CREATE TABLE Account (  
    Acc_id NUMBER PRIMARY KEY,  
    Acc_type VARCHAR2(20), -- Savings / Current  
    Cust_id NUMBER,  
    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id)  
);
```

Insert Sample Data

SQL

```
INSERT INTO Customer VALUES (1, 'LEENA', 'Chennai', '9876543210', 'ABCDE1234F');  
INSERT INTO Customer VALUES (2, 'Ravi', 'Hyderabad', '9123456780', 'PQRSX5678L');
```

INSERT INTO Loan VALUES (101, 50000, 7.5, 1);

INSERT INTO Loan VALUES (102, 30000, 8.0, 1);

INSERT INTO Account VALUES (201, 'Savings', 1);

INSERT INTO Account VALUES (202, 'Current', 2);

2. Constraints and Data Types (6 Marks)

PRIMARY KEY → cust_id, loan_id, acc_id

FOREIGN KEY → Maintains relationship (Loan & Account → Customer)

NOT NULL → Name must be provided

VARCHAR2 → Text fields (name, PAN, phone)

NUMBER → Ids, loan amount, interest

Ensures data integrity and consistency

3. Queries (12 Marks)

(a) Customers Having Both Loan and Account

SQL

SELECT DISTINCT c.name

FROM Customer c

JOIN Loan l ON c.cust_id = l.cust_id

JOIN Account a ON c.cust_id = a.cust_id;

(b) Total Loans and Sum of Loan Amount for "LEENA"

SQL

SELECT COUNT(l.loan_id) AS total_loans,

SUM(l.loan_amount) AS total_amount

FROM Customer c

JOIN Loan l ON c.cust_id = l.cust_id

WHERE c.name = 'LEENA';

(c) Customers Without Loan and Account (Inactive Customers)

SQL

SELECT name

FROM Customer

WHERE cust_id NOT IN (SELECT cust_id FROM Loan)

AND cust_id NOT IN (SELECT cust_id FROM Account);

(d) Add Column for Number of Loans

SQL

ALTER TABLE Customer ADD no_of_loans NUMBER;

(e) Update Number of Loans

SQL

UPDATE Customer c

SET no_of_loans = (

 SELECT COUNT(*)

 FROM Loan l

 WHERE l.cust_id = c.cust_id

);

3. JDBC Program (6 Marks)

Java

Import java.sql.*;

Public class BankDB {

 Public static void main(String[] args) {

 Try {

 // Load Driver

 Class.forName("oracle.jdbc.driver.OracleDriver");

 // Establish Connection

 Connection con = DriverManager.getConnection(

```

        "jdbc:oracle:thin:@localhost:1521:xe",
        "system",
        "password"
    );

    System.out.println("Connected Successfully!");

    // Query Example
    Statement stmt = con.createStatement();
    ResultSet rs = stmt.executeQuery("SELECT * FROM Customer");

    While(rs.next()) {
        System.out.println(
            Rs.getInt(1) + " " +
            Rs.getString(2)
        );
    }

    Con.close();

} catch(Exception e) {
    System.out.println(e);
}
}
}

```

4. Explanation (Extra Marks)

JOIN → Combines multiple tables

COUNT() → Counts number of loans

SUM() → Calculates total loan amount

NOT IN → Finds inactive customers

ALTER TABLE → Modifies table structure

JDBC → Connects Java with database

Q.No: 6

Scenario: Student Information System

A college is developing a Student Information System to manage student personal details

and academic records. The system maintains two tables: one for storing student information

such as registration number, name, date of birth, address, and city; and another for storing

their marks in different subjects along with the total.

During the system's usage, several requirements arise:

- **The administration frequently needs to retrieve the name of a student using a specific**

registration number for verification purposes.

- **Faculty members often search for complete details of students based on their names to**

review academic and personal information.

- **To better reflect academic data usage, the table containing marks information needs to**

be renamed appropriately.

Additionally, the development team is tasked with implementing backend logic to support

academic processing and validations:

- **A procedure is required to calculate both the total and average marks of students for**

performance analysis.

- Another utility program is needed to determine whether a given number satisfies a

mathematical condition related to prime numbers.

Ans).

1. Table Creation and Data Insertion (5 Marks)

Student Table

SQL

```
CREATE TABLE Student (  
    Reg_no NUMBER PRIMARY KEY,  
    Name VARCHAR2(50) NOT NULL,  
    Dob DATE,  
    Address VARCHAR2(100),  
    City VARCHAR2(50)  
);
```

Marks Table

SQL

```
CREATE TABLE Marks (  
    Reg_no NUMBER,  
    Sub1 NUMBER,  
    Sub2 NUMBER,  
    Sub3 NUMBER,  
    Total NUMBER,  
    FOREIGN KEY (reg_no) REFERENCES Student(reg_no)  
);
```

Insert Sample Data

SQL

```
INSERT INTO Student VALUES (101, 'Arun', TO_DATE('2003-04-10','YYYY-MM-DD'), 'MG  
Road', 'Chennai');
```

INSERT INTO Student VALUES (102, 'Priya', TO_DATE('2002-09-15','YYYY-MM-DD'), 'Anna Nagar', 'Chennai');

INSERT INTO Marks VALUES (101, 85, 90, 88, 263);

INSERT INTO Marks VALUES (102, 80, 85, 82, 247);

2. Constraints and Data Types (5 Marks)

PRIMARY KEY → reg_no

FOREIGN KEY → Links Marks with Student

NOT NULL → Name is mandatory

NUMBER → For Ids and marks

VARCHAR2 → For text fields

DATE → For DOB

3. Queries (6 Marks)

(a) Retrieve Name Using Registration Number

SQL

SELECT name

FROM Student

WHERE reg_no = 101;

(b) Retrieve Full Details Using Name

SQL

SELECT *

FROM Student

WHERE name = 'Arun';

(c) Rename Marks Table

SQL

ALTER TABLE Marks RENAME TO Student_Marks;

4. PL/SQL Programs (14 Marks)

(a) Procedure to Calculate Total and Average Marks

SQL

CREATE OR REPLACE PROCEDURE calc_total_avg IS

 Total_sum NUMBER;

 Avg_marks NUMBER;

BEGIN

 SELECT SUM(total), AVG(total)

 INTO total_sum, avg_marks

 FROM Marks;

 DBMS_OUTPUT.PUT_LINE('Total Marks: ' || total_sum);

 DBMS_OUTPUT.PUT_LINE('Average Marks: ' || avg_marks);

END;

/

Execute Procedure

SQL

BEGIN

 Calc_total_avg;

END;

/

(b) Program to Check Prime Number

SQL

DECLARE

 Num NUMBER := 7;

 I NUMBER;

 Flag NUMBER := 0;

BEGIN

 FOR i IN 2..num-1 LOOP

 IF MOD(num, i) = 0 THEN

```

        Flag := 1;
    END IF;
END LOOP;

IF flag = 0 THEN
    DBMS_OUTPUT.PUT_LINE(num || ' is a Prime Number');
ELSE
    DBMS_OUTPUT.PUT_LINE(num || ' is NOT a Prime Number');
END IF;
END;
/

```

5. Explanation (Extra Marks)

WHERE reg_no → Used for specific student lookup

WHERE name → Used for searching by name

ALTER TABLE RENAME → Changes table name

Procedure → Used for repeated calculations

Prime check → Uses loop and modulus logic

Q.No: 7

Scenario: Student Academic Management System

A college is developing a Student Academic Management System to maintain student

personal details and their academic performance. The system stores student information such

as registration number, name, date of birth, address, and city in one table, while another table

maintains marks obtained in different subjects along with the total.

As part of academic and administrative activities:

- The institution requires a consolidated report displaying each student's name along

with their total marks for evaluation purposes.

- During data verification, discrepancies are found in some records, and specific marks

need to be corrected, followed by updating the corresponding total values.

- At a later stage, due to system cleanup or reinitialization, all records from a student-related table along with its allocated storage space must be completely removed.

In addition to database operations, the development team is working on backend logic and

application integration:

- A utility program is required to determine whether a given number satisfies a mathematical condition related to prime numbers.

- A Java-based application is being developed, which must establish a connection with

the database and insert new student records into the system.

Ans).

1. Table Creation and Data Insertion (5 Marks)

Student Table

SQL

```
CREATE TABLE Student (  
    reg_no NUMBER PRIMARY KEY,  
    name VARCHAR2(50) NOT NULL,  
    dob DATE,  
    address VARCHAR2(100),  
    city VARCHAR2(50)  
);
```

Marks Table

SQL

```
CREATE TABLE Marks (  
    reg_no NUMBER,  
    sub1 NUMBER,  
    sub2 NUMBER,  
    sub3 NUMBER,  
    total NUMBER,  
    FOREIGN KEY (reg_no) REFERENCES Student(reg_no)  
);
```

Insert Sample Data

SQL

```
INSERT INTO Student VALUES (101, 'Arun', TO_DATE('2003-04-10','YYYY-MM-DD'), 'MG  
Road', 'Chennai');
```

```
INSERT INTO Student VALUES (102, 'Priya', TO_DATE('2002-09-15','YYYY-MM-DD'), 'Anna  
Nagar', 'Chennai');
```

```
INSERT INTO Marks VALUES (101, 85, 90, 88, 263);
```

```
INSERT INTO Marks VALUES (102, 80, 85, 82, 247);
```

2. Constraints and Data Types (5 Marks)

PRIMARY KEY → reg_no uniquely identifies student

FOREIGN KEY → Maintains relation between tables

NOT NULL → Name is mandatory

NUMBER → For IDs and marks

VARCHAR2 → For text fields

DATE → For DOB

3. Queries (6 Marks)

(a) Display Student Name with Total Marks

SQL

```
SELECT s.name, m.total
```

```
FROM Student s
```

JOIN Marks m ON s.reg_no = m.reg_no;

(b) Update Marks and Total

SQL

UPDATE Marks

SET sub1 = 90, sub2 = 95, sub3 = 85,

total = sub1 + sub2 + sub3

WHERE reg_no = 101;

(c) Remove All Records with Storage (Table Deletion)

SQL

DROP TABLE Student;

4. PL/SQL Program (7 Marks)

Program to Check Prime Number

SQL

DECLARE

num NUMBER := 11;

i NUMBER;

flag NUMBER := 0;

BEGIN

FOR i IN 2..num-1 LOOP

IF MOD(num, i) = 0 THEN

flag := 1;

END IF;

END LOOP;

IF flag = 0 THEN

DBMS_OUTPUT.PUT_LINE(num || ' is a Prime Number');

ELSE

DBMS_OUTPUT.PUT_LINE(num || ' is NOT a Prime Number');

```
END IF;  
END;  
/
```

5. JDBC Program (7 Marks)

Java

```
import java.sql.*;
```

```
public class StudentInsert {  
    public static void main(String[] args) {  
        try {  
            // Load Driver  
            Class.forName("oracle.jdbc.driver.OracleDriver");  
  
            // Establish Connection  
            Connection con = DriverManager.getConnection(  
                "jdbc:oracle:thin:@localhost:1521:xe",  
                "system",  
                "password"  
            );  
  
            // Insert Data  
            PreparedStatement ps = con.prepareStatement(  
                "INSERT INTO Student VALUES (?, ?, ?, ?, ?)"  
            );  
  
            ps.setInt(1, 103);  
            ps.setString(2, "Rahul");  
            ps.setDate(3, Date.valueOf("2003-01-20"));
```

```

ps.setString(4, "KPHB");
ps.setString(5, "Hyderabad");

ps.executeUpdate();

System.out.println("Record Inserted Successfully!");

con.close();

} catch(Exception e) {
    System.out.println(e);
}
}
}

```

6. Explanation (Extra Marks)

JOIN → Combines student and marks data

UPDATE → Corrects marks and recalculates total

DROP TABLE → Deletes table + structure + storage

PL/SQL → Used for logical operations (prime check)

JDBC → Used for database connection and insertion

Q.No: 8

Scenario: Customer and Loan Management System

A bank is developing a Customer and Loan Management System to efficiently handle

customer information and loan-related details.

The system maintains a customer table that stores details such as customer ID, name, address,

phone number, and PAN number. Another table is used to store loan information including

loan ID, loan amount, interest, associated customer ID, and branch name.

As part of banking operations and reporting:

- The management requires a sorted view of all loan records based on the loan amount

to analyze high-value and low-value loans.

- The bank also needs to identify and list all the branches that are involved in providing

loans.

- For financial analysis, the system should be able to filter and identify customers who

have taken loans within a specific amount range.

In addition to database operations, the development team is implementing utility programs:

- A program is required to process textual input and determine whether it satisfies a

specific condition related to symmetry in characters.

Furthermore, the application team is working on system integration:

- A Java-based application must be developed to establish a connection with the database to perform operations on customer and loan data.

Ans).

1. Table Creation and Data Insertion (5 Marks)

Customer Table

SQL

```
CREATE TABLE Customer (  
    cust_id NUMBER PRIMARY KEY,  
    name VARCHAR2(50) NOT NULL,  
    address VARCHAR2(100),  
    phone VARCHAR2(15),
```

```
pan_no VARCHAR2(20)
);
```

Loan Table

SQL

```
CREATE TABLE Loan (
    loan_id NUMBER PRIMARY KEY,
    loan_amount NUMBER,
    interest NUMBER,
    branch_name VARCHAR2(50),
    cust_id NUMBER,
    FOREIGN KEY (cust_id) REFERENCES Customer(cust_id)
);
```

Insert Sample Data

SQL

```
INSERT INTO Customer VALUES (1, 'Ravi', 'Chennai', '9876543210', 'ABCDE1234F');
INSERT INTO Customer VALUES (2, 'Priya', 'Hyderabad', '9123456780', 'PQRSX5678L');
```

```
INSERT INTO Loan VALUES (101, 50000, 7.5, 'Chennai', 1);
```

```
INSERT INTO Loan VALUES (102, 30000, 8.0, 'Hyderabad', 2);
```

2. Constraints and Data Types (5 Marks)

PRIMARY KEY → cust_id, loan_id

FOREIGN KEY → Links Loan with Customer

NOT NULL → Name cannot be null

NUMBER → Loan amount, interest, IDs

VARCHAR2 → Name, address, branch

Ensures data integrity and consistency

3. Queries (6 Marks)

(a) Sort Loan Records by Loan Amount

SQL

SELECT *

FROM Loan

ORDER BY loan_amount ASC;

(b) List All Branches Providing Loans

SQL

SELECT DISTINCT branch_name

FROM Loan;

(c) Customers with Loan Amount in Range

SQL

SELECT c.name, l.loan_amount

FROM Customer c

JOIN Loan l ON c.cust_id = l.cust_id

WHERE l.loan_amount BETWEEN 30000 AND 60000;

4. PL/SQL Program (7 Marks)

Program to Check Palindrome (Symmetry in Characters)

SQL

DECLARE

 str VARCHAR2(50) := 'madam';

 rev VARCHAR2(50) := '';

 i NUMBER;

BEGIN

 FOR i IN REVERSE 1..LENGTH(str) LOOP

 rev := rev || SUBSTR(str, i, 1);

 END LOOP;

 IF str = rev THEN

 DBMS_OUTPUT.PUT_LINE('String is Palindrome');

```
ELSE
    DBMS_OUTPUT.PUT_LINE('String is NOT Palindrome');
END IF;
END;
/
```

5. JDBC Program (7 Marks)

Java

```
import java.sql.*;

public class BankDB {
    public static void main(String[] args) {
        try {
            // Load Driver
            Class.forName("oracle.jdbc.driver.OracleDriver");

            // Establish Connection
            Connection con = DriverManager.getConnection(
                "jdbc:oracle:thin:@localhost:1521:xe",
                "system",
                "password"
            );

            System.out.println("Connected Successfully!");

            // Display Loan Data
            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM Loan");
```

```

while(rs.next()) {
    System.out.println(
        rs.getInt(1) + " " +
        rs.getDouble(2)
    );
}

con.close();

} catch(Exception e) {
    System.out.println(e);
}
}
}

```

6. Explanation (Extra Marks)

ORDER BY → Sorts loan data

DISTINCT → Removes duplicate branch names

BETWEEN → Filters loan amount range

Palindrome logic → Checks symmetry of string

JDBC → Enables Java–Database connectivity

Q.No: 9

Scenario: Employee Management and Payroll System

A company is developing an Employee Management and Payroll System to organize employee records and departmental information in a structured database.

Initially, the system is designed to include:

- **A table to store employee details such as employee number, name, date of birth, address, date of joining, mobile number, department number, and salary.**

- Another table to maintain department information including department number, department name, and location.

As the organization begins using the system, several new requirements emerge:

- The HR department wants to analyze employee experience by calculating the number

of months each employee has worked, based on their date of joining and the current

date.

- To ensure data integrity and uniqueness of employee records, the database structure

needs to enforce a unique identification constraint on employee numbers.

- For payroll processing, the system should include a PL/SQL function that retrieves

and displays the salary of a specific employee when required.

- Additionally, the software development team is building a Java-based application that interacts with the database. This application must be capable of establishing a connection using JDBC and inserting employee records into the database.

Ans).

1. Table Creation and Data Insertion (5 Marks)

Department Table

SQL

```
CREATE TABLE Department (  
    Dept_no NUMBER PRIMARY KEY,  
    Dept_name VARCHAR2(50),  
    Location VARCHAR2(50)  
);
```

Employee Table

SQL

```
CREATE TABLE Employee (  

```

```
Emp_no NUMBER PRIMARY KEY,  
Name VARCHAR2(50) NOT NULL,  
Dob DATE,  
Address VARCHAR2(100),  
Doj DATE,  
Mobile VARCHAR2(15),  
Dept_no NUMBER,  
Salary NUMBER,  
FOREIGN KEY (dept_no) REFERENCES Department(dept_no)  
);
```

Insert Sample Data

SQL

```
INSERT INTO Department VALUES (10, 'HR', 'Chennai');
```

```
INSERT INTO Department VALUES (20, 'IT', 'Hyderabad');
```

```
INSERT INTO Employee VALUES (101, 'Ravi', TO_DATE('1995-05-10','YYYY-MM-DD'),  
'Chennai', TO_DATE('2020-01-15','YYYY-MM-DD'), '9876543210', 10, 30000);
```

```
INSERT INTO Employee VALUES (102, 'Priya', TO_DATE('1996-08-20','YYYY-MM-DD'),  
'Hyderabad', TO_DATE('2021-06-10','YYYY-MM-DD'), '9123456780', 20, 35000);
```

2. Constraints and Data Types (5 Marks)

PRIMARY KEY → emp_no, dept_no (unique identification)

FOREIGN KEY → Links Employee with Department

NOT NULL → Name is mandatory

VARCHAR2 → Name, address, mobile

NUMBER → Salary, Ids

DATE → DOB and DOJ

Ensures data integrity and uniqueness

3. Queries (6 Marks)

(a) Calculate Employee Experience in Months

SQL

```
SELECT name, MONTHS_BETWEEN(SYSDATE, doj) AS experience_months  
FROM Employee;
```

(b) Ensure Unique Employee Number (Already via PK)

SQL

-- Already enforced using PRIMARY KEY constraint on emp_no

4. PL/SQL Function (7 Marks)

Function to Get Employee Salary

SQL

```
CREATE OR REPLACE FUNCTION get_salary(p_emp_no NUMBER)
```

```
RETURN NUMBER
```

```
IS
```

```
    v_salary NUMBER;
```

```
BEGIN
```

```
    SELECT salary INTO v_salary
```

```
    FROM Employee
```

```
    WHERE emp_no = p_emp_no;
```

```
    RETURN v_salary;
```

```
END;
```

```
/
```

Call Function

SQL

```
SELECT get_salary(101) FROM dual;
```

5.JDBC Program (7 Marks)

Java

```
Import java.sql.*;
```

```
Public class EmployeeInsert {
```

```
    Public static void main(String[] args) {
```

```
        Try {
```

```
            // Load Driver
```

```
            Class.forName("oracle.jdbc.driver.OracleDriver");
```

```
            // Establish Connection
```

```
            Connection con = DriverManager.getConnection(
```

```
                "jdbc:oracle:thin:@localhost:1521:xe",
```

```
                "system",
```

```
                "password"
```

```
            );
```

```
            // Insert Employee Record
```

```
            PreparedStatement ps = con.prepareStatement(
```

```
                "INSERT INTO Employee VALUES (?, ?, ?, ?, ?, ?, ?, ?)"
```

```
            );
```

```
            Ps.setInt(1, 103);
```

```
            Ps.setString(2, "Kiran");
```

```
            Ps.setDate(3, Date.valueOf("1997-03-15"));
```

```
            Ps.setString(4, "Bangalore");
```

```
            Ps.setDate(5, Date.valueOf("2022-02-01"));
```

```
            Ps.setString(6, "9988776655");
```

```
            Ps.setInt(7, 10);
```

```
            Ps.setInt(8, 40000);
```

```
Ps.executeUpdate();

System.out.println("Employee Inserted Successfully!");

Con.close();

} catch(Exception e) {
    System.out.println(e);
}
}
}
```

6. Explanation (Extra Marks)

MONTHS_BETWEEN → Calculates experience

PRIMARY KEY → Ensures uniqueness of employee

Function → Returns salary for given employee

JDBC → Connects Java application with database

PreparedStatement → Used for secure insertion